



Data Mining Project

SENTIMENT ANALYSIS OF STEAM GAME REVIEWS



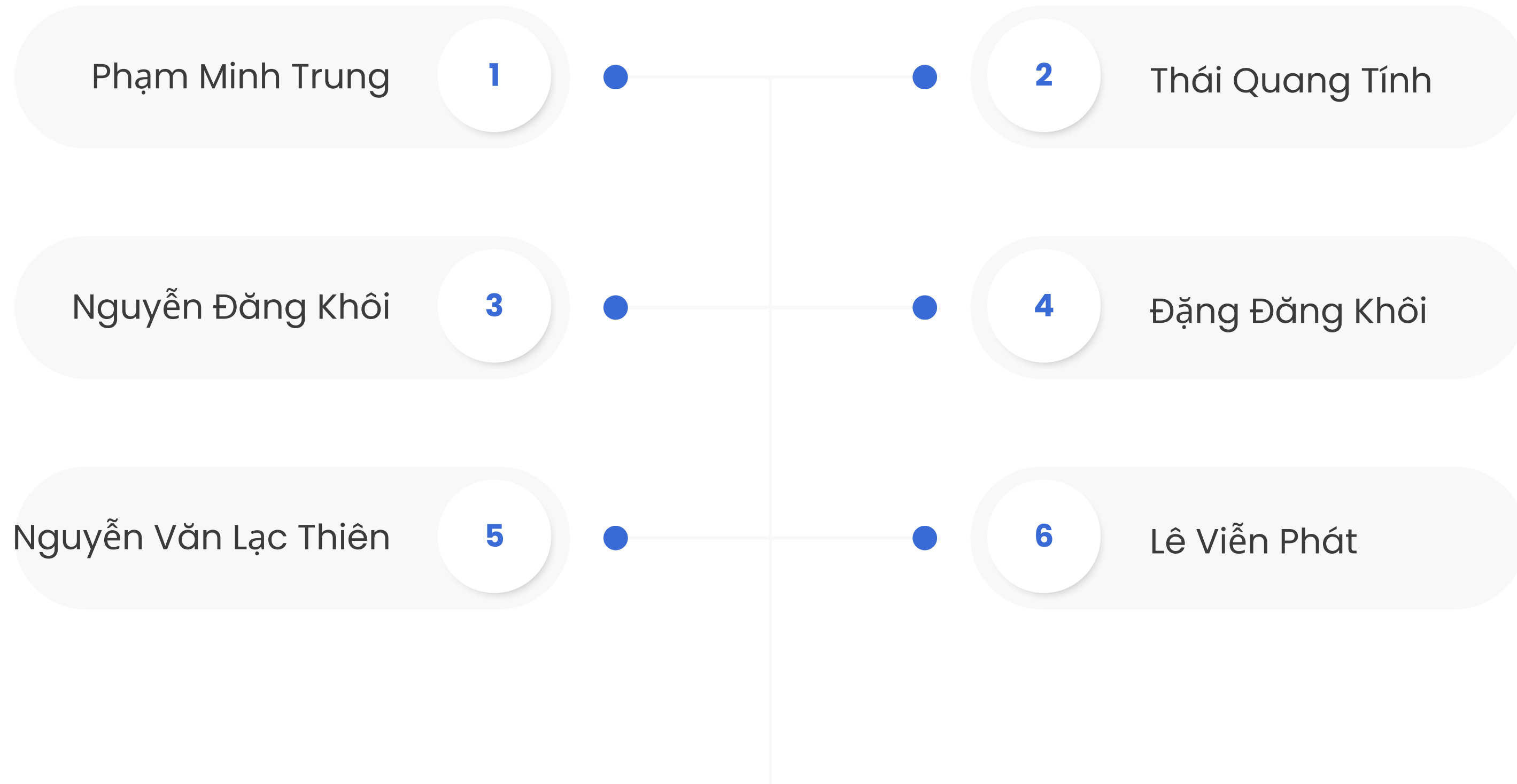


Outline

- **1** Introduction
- **2** Implementations
- **3** Improvements
- **4** Evaluation
- **5** Conclusion



Team Member





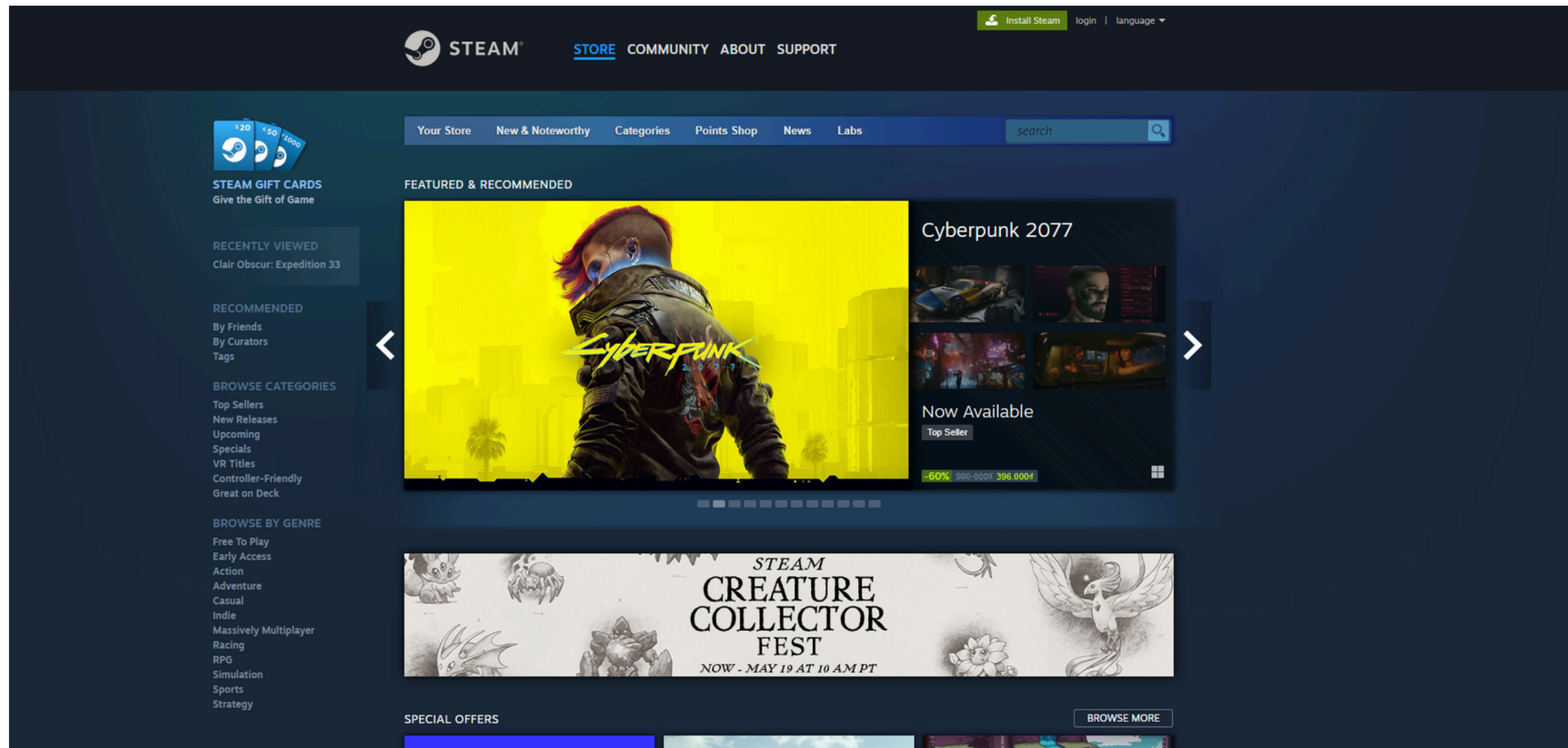
Introduction

Overview

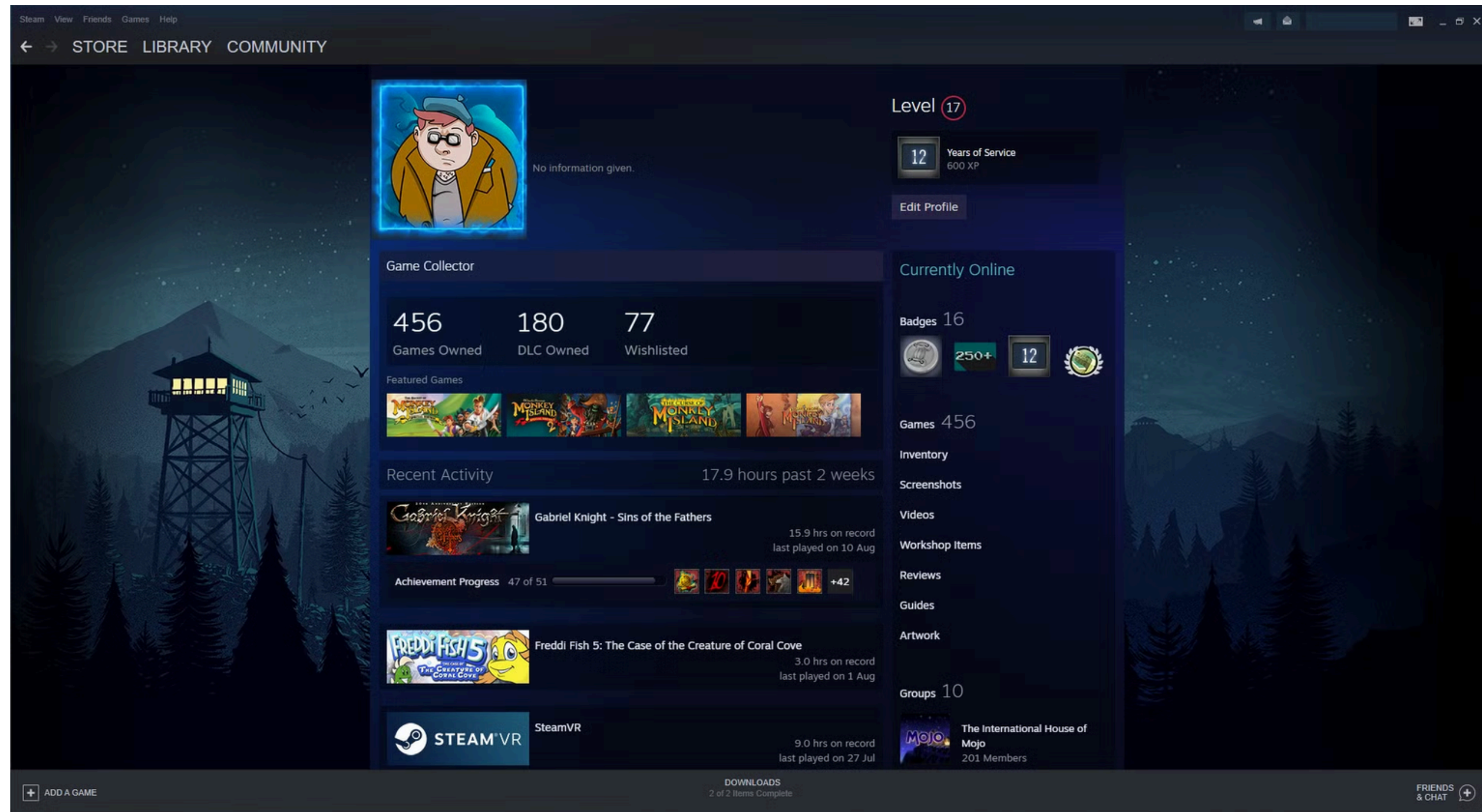


What is steam ?

Overview

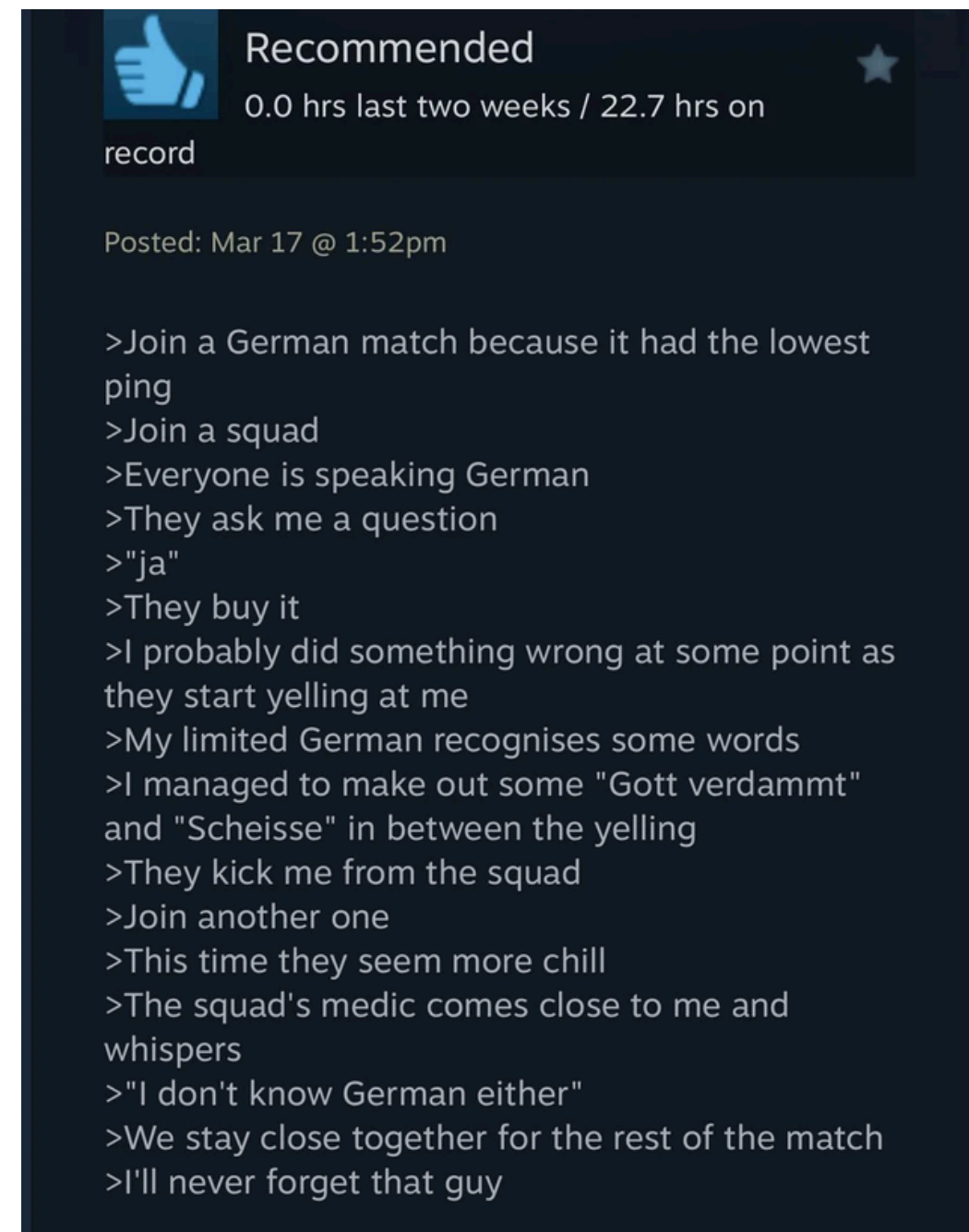
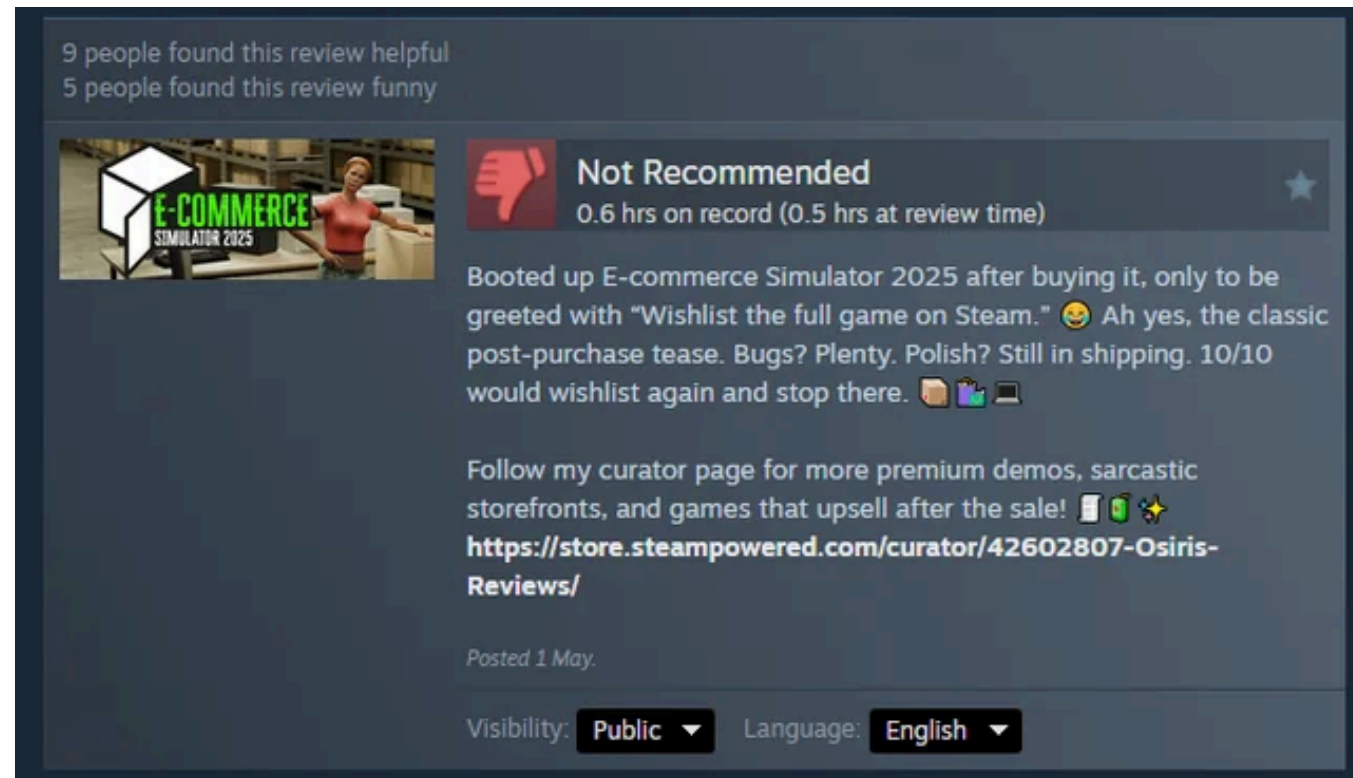


Frontpage of Steam



An example of Steam social network

Overview



Users sharing their experiences



Implementations



Data Preparation & Processing

1

Preparation

2

Processing

3

Feature Selection



Preparation

1

Crawl Data

Tools:

BeautifulSoup
Concurrent
Request
Pandas

2

Clean Raw Data

Applied a filtering process based on game titles, removing games containing keywords such as "**theme**", "**lite**", "**packs**", and "**tools**"



Processing

1

Game Data

Consists of 50,795 games with 11 attributes with mixture of Categorical and Numerical Variables

Characteristics: Wrong data types, missing values and class variations

2

Game Review

Consists of 322,003 reviews with 9 attributes with mixture of Categorical and Numerical Variables

Characteristics: Missing values, data types variations such as time and boolean



Processing

1

Game Data

#	Column	Non-Null Count	Dtype
--	-----	-----	-----
0	app_id	50795 non-null	int64
1	name	50795 non-null	object
2	tags	50795 non-null	object
3	popular_tags	50795 non-null	object
4	all_reviews	46548 non-null	object
5	all_percent	50795 non-null	object
6	all_count	30779 non-null	float64
7	release_date	50795 non-null	object
8	developer	50462 non-null	object
9	publisher	50195 non-null	object
10	price	50795 non-null	object
11	description	50795 non-null	object

2

Game Review

0	app_id	322003 non-null	int64
1	recommendationid	322003 non-null	int64
2	review	322003 non-null	object
3	voted_up	322003 non-null	bool
4	timestamp_created	322003 non-null	int64
5	playtime_forever	322003 non-null	float64
6	weighted_vote_score	322003 non-null	float64
7	votes_up	322003 non-null	int64
8	steamid	322003 non-null	int64



Processing

1

Convert to Numerical data

Convert wrong data type columns into numerical data such as **'price'** and **'all_percent'**

2

Handling missing values

Categorical: Missing values are replaced with **'Unknown'** or **'No reviews'**

Numerical: Handling by filling '0' with missing values or **'Free'**

3

Distribution Cluster

Categorical: Group data like release_date and tags based on their real-world significance

Numerical: Apply meaningful K of cluster suggested by the community

4

Encoding categorical data

Categorical: Apply a mapping technique to convert them into numeric values.

Numerical: Use clustering results to assign appropriate numeric values.



Feature Selection

1

Game Data

Columns such as **name**, **publisher**, **developer**, and **popular_tags** were dropped since they are not contributing to the classification task

2

Game Review

Columns such as **timestamp_created**, **playtime_forever**, **weighted_vote_score**, and **votes_up** were discarded, as they were not relevant for sequence mining.



Model Development

1. OneR

2. Naive Bayes

3. J48

OneR, NaiveBayes and J48

OneR

- Uses one single attribute
- Model complexity: very low
- Extremely fast and lightweight

NaiveBayes

- Uses all attributes
- Model complexity: moderate
- fast and scalable to large datasets

J48

- Builds a decision tree by selecting best splits recursively
- Model complexity: Medium to high
- Slower than OneR/NB on large data

Result

Metrics	J48	Naive Bayes	OneR
Correctly Classified Instances	72.3481	71.9814	70.148
Incorrectly Classified Instances	27.6519	28.0186	29.852
Kappa statistic	0.6511	0.6464	0.622
Mean absolute error	0.0692	0.0692	0.0543
Root mean squared error	0.1866	0.1864	0.233
Relative absolute error	46.85 %	46.89 %	36.76 %
Root relative squared error	68.68 %	68.61 %	85.74 %



Association Rules

Goal:

Find rules for games that users likely vote up together

Apriori

Generates large number of candidate itemsets
Requires multiple scans
--> uses more memory

FPGrowth

Compresses into FP-tree
Requires two scans

Result (10 Best Rules)

Top 10 Best Association Rules:

Antecedent Name	Consequent Name	Lift	Confidence	Support
Arena of the Titans	Time Over	115.74	0.86	0.0057
Time Over	Arena of the Titans	115.74	0.77	0.0057
I Love You Freddy, Ancient Ruins	Falco Xolon	93.63	0.85	0.0041
Falco Xolon	I Love You Freddy, Ancient Ruins	93.63	0.45	0.0041
I Love You Freddy, Falco Xolon	Ancient Ruins	91.23	0.85	0.0041
Ancient Ruins	I Love You Freddy, Falco Xolon	91.23	0.44	0.0041
Falco Xolon	Ancient Ruins	90.39	0.84	0.0076
Ancient Ruins	Falco Xolon	90.39	0.82	0.0076
Arena of the Titans	Ancient Ruins	72.83	0.68	0.0045
Ancient Ruins	Arena of the Titans	72.83	0.49	0.0045



Improvements



Improvements

- **1** Dataset Improvement
- **2** Model Improvement



Dataset Improvement

Dataset Improvement

a. Numerical Data

Use Elbow Method to find best k for K-Means Cluster. Best k will be applied to the dataset to normalize the variables.

b. Tags Clustering

Use TF-IDF Vectorizer to find the similarity between tags, then use Elbow Method to find optimal k.

c. Removing Outliers

Visualize the dataset to track outliers, either remove or replace with optimal values.



Model Improvement

Model Improvement

a. Data Description

- **File used:** steam game data
- **Attributes used:** from column 3 to 8 in the dataset
- **Class attribute:** all reviews encoded

b. Preprocessing Steps

- **Filtering:** Used Weka's Remove filter to keep only relevant attributes
- **Class Index Set:** Set to the index of all_reviews_encoded

c. J48 Classifier Settings

- **Classifier:** Used Weka's J48 (Weka implementation of C4.5)
- **Parameters:** Set options "-C", "0.5", "-M", "10"

Model Improvement

d. Evaluation Strategy

- **Method:** 10-fold cross-validation
- **Random Seed:** 1
- **Evaluation Metrics:** Accuracy, summary statistics, class details and confusion matrix

e. Results(Before Improvement)

- **Key Observations are:**
 - The model's performance is decent(Accuracy is 72,35%), can be improved more.
 - Interesting insides: class 5 and 8 is handled good with prediction nearly 100%, while class 2 and 8 has 0 prediction accuracy.
 - The tree is quite complex with 251 leaves 292 nodes

f. Results(After Improvement)

- **Key Observations are:**
 - The model's performance is significant increase (Accuracy is 79,14%)
 - Interesting insides: The model has smaller tree with 63 leaves 76 nodes
 - Most of the classes has improved the prediction accuracy

Model Improvement

f.

Result Comparison

Metrics	J48	Naive Bayes	OneR
Correctly Classified Instances	79.13 %	78.61 %	77.44 %
Incorrectly Classified Instances	20.86 %	21.39 %	22.56 %
Kappa statistic	0.708	0.6983	0.677
Mean absolute error	0.0699	0.0704	0.0564
Root mean squared error	0.1872	0.1879	0.2375
Relative absolute error	38.75 %	39.04 %	31.26 %
Root relative squared error	62.32 %	62.55 %	79.07 %

- Apply K-means clustering to group the attribute values in the dataset.
- Elbow method to choose the right number of clusters (k)
- J48’s accuracy increased from 72.35% to 79.14%, which is about a 6% improvement.
- Accuracy go up, but other metrics also improved across the board.
- J48 clearly more effective than its original form and stronger than both OneR and Naive Bayes overall.



Evaluation

Model Evaluation

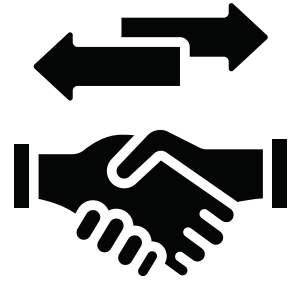


Model	Accuracy	F1 Score	Runtime
Naive Bayes	78.62%	0.78	1.17 s
OneR	77.44%	0.77	1.00 s
J48	79.12%	0.79	2.46 s

Table 4.2. Performance Metrics

- J48 most accurate, slower runtime, gave the highest accuracy and F1 score.
- Naive Bayes balances accuracy & speed with fast execution.
- OneR simple and fast baseline with decent results.
- Association Rule gave insights - not performance metrics.

Analysis



Trade-offs:

- J48: Accurate but complex and slower.
- Naive Bayes: Efficient but strong assumptions.
- OneR: Fast, simple, and less predictive.
- Association Rules: Pattern discovery, not prediction.

Challenges:



- Major class imbalance (positive reviews dominate).
- Noisy text: slang, emojis, informal language.
- Scalability: dataset size stressed memory & runtime.
- Hyperparameter tuning was time-consuming.



Conclusion

Key findings

- Data preprocessing is just as important as model selection
- Trying out different classifiers is helpful
- Evaluation through cross-validation is essential to get reliable performance metrics
- Class imbalance can impact the final results alot !

Future works

- Apply Class Balancing Techniques
- Explore Deep Learning
- Expand Feature Set
- Deploy as a Web App



**Thank
You**

